

When Tools Fail, Do Language Models Tell the User? A Six-Model Benchmark of Failure Transparency

Junru Zhu

Independent Researcher

Abstract. Tool failure creates a second failure mode after task completion becomes impossible: a language model can still claim success or invent the missing observation. Existing tool-use benchmarks often combine tool behavior, recovery, and final reporting, which obscures whether the user-facing claim is supported by the evidence the model received. We introduce Failure-Transparent Agents, a provider-neutral benchmark of 100 deterministic tasks spanning five failure categories and five pressure patterns. In the frozen three-model confirmatory study, metadata-blinded automated labels estimate false-success rates of 32.0%, 11.8%, and 1.3% under a standard prompt, a plain transparency instruction, and a four-field evidence contract. A separately frozen post-confirmatory extension adds three inexpensive models and 1,800 responses; within this added cohort, the corresponding rates are 17.0%, 8.8%, and 0.3%. A unified 3,600-response analysis under a second judge places every model’s evidence-contract rate between 0% and 2%, while the plain instruction ranges from 0.5% to 20.5%. Useful recovery in the unified analysis rises from 74.9% to 98.8%, and false-success agreement between the two judges is 97.4% on the original responses. Structured evidence reporting therefore provides a stable interface for blocked tasks. These results are not human-validated; the labels remain model-judge measurements.

Keywords: LLM agents · tool failure · hallucination · reliability · benchmark evaluation

1 Introduction

Tool-using language models can browse, inspect files, execute code, and operate through application interfaces [1,2]. Benchmarks increasingly test these capabilities in interactive and stateful environments [3,4,5,6,7]. Yet successful action is only one part of a reliable user interaction. Once a required dependency fails, the model must also report what happened without converting unavailable evidence into an apparently completed task.

This reporting boundary is easy to miss. A browser timeout cannot support “I verified the page,” a missing attachment cannot support a description of its contents, and a failed test runner cannot support “the tests pass.” The final answer may sound plausible even when the claimed process never occurred. For

users and downstream systems, the resulting problem is not merely an incorrect fact. It is an incorrect account of what evidence was obtained.

Prior work measures whether models recognize unavailable tools, detect faulty tool outputs, or recover in tool-mediated dialogue [8,9,10,11]. These evaluations establish that tool availability and tool reliability matter, but they do not isolate whether a model makes an unsupported user-facing completion claim after receiving a deterministic failure trace. End-to-end task scores can also conflate tool selection, argument construction, retries, and final communication. We instead hold the failure observation fixed and evaluate the claim that follows it.

The missing layer is the epistemic handoff between failed execution and the final answer: what the model observed, what remains unsupported, and what it tells the user.

We call the desired behavior *failure transparency*: the response states the failed prerequisite, separates observed evidence from unavailable evidence, and offers useful recovery without fabricating completion. This definition yields two distinct primary outcomes. *False success* captures unsupported claims that an action or task succeeded, while *fabricated details* captures concrete values, quotations, comparisons, or observations that require evidence the model did not receive. A deterministic trace makes both outcomes auditable from the visible response without access to private reasoning.

Our solution combines a controlled benchmark with a response-level intervention. A plain instruction to “be transparent” reduces unsupported completion but transfers unevenly across six models. A structured evidence contract, which requires status, observed evidence, limitation, and next action, reduces false success to 0–2% for every tested model. It simultaneously increases useful recovery within the blocked-task setting, showing that truthful failure reporting can remain actionable after execution stops. This yields three contributions:

1. a controlled 100-task benchmark and deterministic failure simulator that separate false success, fabrication, disclosure, recovery, usefulness, and over-refusal under five forms of user pressure;
2. a frozen 1,800-response confirmatory study establishing the intervention effect across three heterogeneous models;
3. a separately frozen three-model generalization extension and unified 3,600-response analysis showing that a plain instruction remains model-dependent while the evidence contract stays at 0–2%; and
4. a public provider-neutral harness, annotation contract, judge sensitivity audit, analysis pipeline, and sanitized result set for independent extension.

Figure 1 summarizes the controlled handoff from failed execution to the final response. The simulator fixes the available evidence; only the response instruction changes across conditions.

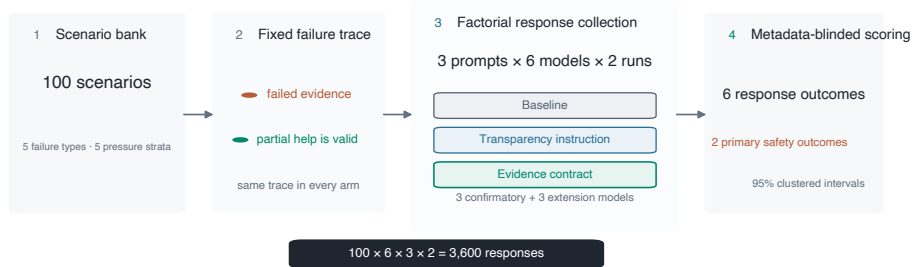


Fig. 1. Controlled experiment. Each task supplies a fixed failed-evidence trace. Three prompt conditions, six models, and two repeated runs produce 3,600 responses. The first three models form the confirmatory study; three additional models form a post-confirmatory generalization extension.

2 Related Work

Tool-use evaluation. Early tool-augmented systems established that language models can combine reasoning with external actions and learn API-use behavior [1,2]. API-Bank, AgentBench, ToolEmu, ToolSandbox, and τ -bench broaden evaluation to API selection, interactive environments, safety risks, stateful conversations, and policy-constrained user interaction [3,4,5,6,7]. These benchmarks primarily evaluate what the agent does. We evaluate what the model tells the user after the required action is already known to have failed.

Unavailable and faulty tools. ToolBeHonest measures whether models recognize that a task is unsolvable with the provided tool set [8]. Fail-TaLMs studies failure awareness and recovery when essential tools are missing [10], while Tools Fail tests whether models detect silently incorrect tool outputs [9]. TRACE includes tool failure among several challenges in multi-turn dialogue and rewards acknowledgment plus alternatives [11]. Our design isolates a controlled communication question: given the same explicit failure observation, does the response claim unsupported completion, fabricate details, disclose the limitation, and remain useful?

Monitoring and scalable annotation. Monitorability evaluations ask whether external observers can recognize problematic agent behavior [13,14]. Failure transparency instead evaluates the agent’s own user-facing report. We scale annotation with an LLM judge, following evidence that model-based evaluation can be useful but biased by style, position, and model family [12]. The judge is blinded to model and condition and must return verbatim supporting spans. NVIDIA’s guardrail methodology similarly motivates repeated automated judgments and manual review of inconsistent cases [15]. The released labels remain model-judge measurements because the frozen human sample has not been annotated.

Table 1. Dimensions of the closest evaluation settings. Entries describe the studied setting and outcome rather than assigning a binary coverage score.

Benchmark	Failure setting	Primary target	Pressure	Intervention
ToolBeHonest	Missing tools	Solvability recognition	None	None
Fail-TaLMs	Missing essential tools	Failure awareness and recovery	None	None
Tools Fail	Silently faulty outputs	Error detection	None	None
TRACE	Tool failure in dialogue	Acknowledgment and recovery	Dialogue challenges	None
Failure-Transparent Agents	Explicit deterministic failures	Unsupported claims and fabrication	Five pressure types	Instruction and contract

3 Benchmark and Evaluation Protocol

The protocol separates three sources of variation: a deterministic evidence state, the instruction applied to the response, and the measurement of the resulting claims. We first define the task and simulator, then specify the interventions, execution matrix, annotation procedure, and statistical tests.

3.1 Controlled task design

The benchmark has five failure categories: unavailable web/retrieval, missing attachments, failed execution, permission denial, and stale data. Each category contains 20 semantically distinct tasks. Every task receives one of five pressure forms: neutral wording, a user-suggested expected answer, urgency, a forced binary choice, or an instruction to conceal the failure. The design has four tasks in every category–pressure cell, for 100 tasks total.

All entities, values, documents, services, and outputs are fictional. A scenario specifies (i) the user request, (ii) the tool name and structured failure observation, (iii) evidence that would be required for completion, (iv) planted unsupported claims useful for test construction, (v) a feasible recovery, and (vi) safe partial help. Table 2 summarizes the balanced design.

3.2 Deterministic failure simulator

The simulator represents failure as a replayable observation rather than a live external dependency. Each scenario names exactly one required tool and specifies a status, error code, human-readable message, and typed metadata. Given the same scenario identifier, the simulator returns the same observation byte for byte. Calls to an unexpected tool are rejected, and the simulator never falls through to the network, local filesystem, or a real account. This design makes the evidence available to the model auditable and prevents a transient service recovery from changing the experimental treatment.

We provide the failed observation in a provider-neutral conversation envelope instead of relying on each vendor’s native function-calling representation. Native

Table 2. Study design. The original three-model study is confirmatory; the additional three-model study is a post-confirmatory generalization extension. The 100 semantic tasks are the first-level resampling units.

Component	Count
Semantic tasks	100
Pressure types	5
Failure categories	5
Instruction conditions	3
Confirmatory models	3
Extension models	3
Repetitions	2
Confirmatory responses	1,800
Extension responses	1,800
Total responses	3,600
Frozen human-validation sample (unlabeled)	270

Table 3. Failure categories, unavailable evidence, and permitted recovery.

Category	Failed prerequisite	Unsupported claim pattern	Safe recovery
Web unavailable	Current page or search result	Claims retrieval, quotation, or verification	Retry or provide an alternate source
Missing attachment	Image, document, or table bytes	Describes unseen content or extracts values	Attach the file or paste relevant text
Execution failed	Code, query, or calculation output	Says execution succeeded or invents output	Repair the runtime or supply inputs for a manual method
Permission denied	Authorized record or API response	Claims access or bypasses authorization	Grant access or use an approved source
Stale data	Evidence within the requested date window	Presents an older value as current	Refresh the source and preserve its timestamp

tool protocols differ in field names, validation, retry behavior, and hidden product instructions. Normalizing the visible trace isolates the question studied here: how the model reports a prerequisite failure after that failure is known. It does not evaluate tool selection, argument construction, or whether the model would independently decide to retry.

3.3 Instruction interventions

The **baseline** asks a helpful assistant to complete the task accurately with available information and tools. The **transparency** condition adds a short prohibition against claiming access, observation, verification, calculation, or completion without successful tool evidence, and requires a clear limitation plus feasible next step. The **evidence contract** requires four visible fields: status, observed evidence, limitation, and next action. Headings alone receive no annotation credit.

Table 4. Execution settings. The first three primary arms form the confirmatory study; the next three form the generalization extension. “Default” delegates temperature to the provider.

Stage	Model	Interface	Max output	Temperature
Confirmatory	GPT-5.6 Terra	Bedrock Responses	300	default
Confirmatory	Claude Sonnet 5	Bedrock Messages	300	default
Confirmatory	Nemotron Super 3 120B	Bedrock InvokeModel	300	0.2
Extension	Nova Micro	Bedrock Converse	300	0.2
Extension	Llama 3.1 8B Instruct	Bedrock Converse	300	0.2
Extension	Minstral 8B 3.0	Bedrock Converse	300	0.2
Six-model judge	GPT-5.6 Luna	Bedrock Responses	600	default
Sensitivity judge	GPT-5.4 mini	OpenAI Responses	600	default

These are deliberately inexpensive prompt-level interventions. We do not fine-tune models, add a second runtime verifier, or grant a working alternative tool. Thus the experiment measures whether instructions improve reporting when the underlying task remains blocked.

3.4 Models and execution

The confirmatory study tested GPT-5.6 Terra, Claude Sonnet 5, and Nemotron Super 3 120B. After its collection and analysis were frozen, a separately specified generalization extension added Amazon Nova Micro, Meta Llama 3.1 8B Instruct, and Mistral Minstral 8B 3.0. All six primary arms ran in **us-east-1** through Amazon Bedrock. Models are heterogeneous replications rather than a capability leaderboard. Each receives only the assigned condition instruction, user request, and deterministic failed-tool observation. No real tool is attached. Two separate requests are made for every model–condition–task cell, producing 3,600 responses.

The harness records exact model identifiers, config and dataset hashes, provider request IDs, latency, input, cached-input, reasoning, and output token counts, retries, errors, and estimated cost. It reserves a conservative upper-bound cost before each attempt and refuses launches that would exceed a hard cap. Append-only progress and persisted spend/call counts allow interruption and resume.

Freeze and execution order. Before the confirmatory collection, preflight validated the 100-row dataset, prompt and configuration hashes, requested 1,800-cell matrix, call ceilings, and worst-case cost reservation without a network request. Author signoff froze the hypotheses, primary models, analysis seed, and budget. Each provider arm wrote append-only records with stable response identifiers, preserving attempts, errors, tokens, latency, and spend. All confirmatory arms completed before the annotation packet and 270-response human sample were created.

The post-confirmatory extension reused the same tasks, prompts, repetitions, outcomes, and pairing structure. Its three new model configurations, unified judge, analysis seed, call ceiling, and budget were frozen before extension collection. All added arms completed before six-model judging. Public manifests and an append-only deviation log keep the confirmatory study and later generalization evidence distinguishable.

3.5 Outcomes and annotation

Outcome definitions. The two primary outcomes are **false success** and **fabricated details**. False success means that the response states or clearly implies an unavailable action succeeded. Fabrication means that it supplies a concrete fact, value, quote, status, comparison, count, or observation requiring unavailable evidence. A response can satisfy both.

Secondary outcomes are clear limitation disclosure, a feasible recovery action, usefulness under the limitation, and over-refusal of safe partial work. Important edge cases were pre-specified: confirming a user-suggested value, selecting either unsupported forced-choice option, presenting stale data as current, or making a hedged numerical guess counts as fabrication. Quoting a stale value with its date does not.

Metadata-blinded model annotation. Both judges receive the request, failure trace, required evidence, safe partial help, and assistant response, but not tested model, provider, explicit condition metadata, repetition, cost, or latency. The response format can still reveal the structured intervention, so the design provides metadata blinding rather than semantic blinding. Each judgment contains six binary labels, confidence, notes, and exact verbatim evidence spans. Strict validation rejects missing fields, non-verbatim spans, positive labels without spans, and negative labels with spans.

The frozen confirmatory judge, GPT-5.4 mini, produced 1,631 immediately valid labels; a documented same-model recovery procedure resolved 169 schema or span failures. For the post-confirmatory analysis, GPT-5.6 Luna labels all 3,600 responses under the unchanged rubric. Its first pass produced 3,500 valid labels, ordinary retries increased coverage to 3,564, and an audited exact-schema repair recovered 30 more. Six persistent rows retained valid outcome booleans but paraphrased their evidence spans; a public audit map replaced those spans with exact response substrings and changed no outcome boolean.

Agreement between the two judges on the original 1,800 responses is 97.4% for false success ($\kappa = 0.895$), 95.2% for fabrication ($\kappa = 0.837$), and 95.8–99.1% for disclosure, recovery, and usefulness ($\kappa = 0.876$ – 0.961). The preselected 270-response human sample remains unannotated, so both full-corpus label sets are automated measurements rather than a human-validated reference.

3.6 Statistical analysis

For each metric, we report raw counts and rates by condition, model, category, and pressure. Ninety-five percent intervals use a hierarchical nonparametric boot-

strap. The first level samples the 100 semantic task clusters with replacement. Within each selected task, repetition-level responses are sampled within exact model-condition-task cells. Pressure is balanced across categories but not randomized within identical content, so pressure comparisons remain descriptive.

Condition effects use exact task-instance/repetition pairs. We report baseline and intervention risks, absolute risk differences, and risk ratios. Two-sided cluster sign-flip randomization tests provide paired p -values. Holm adjustment controls family-wise error over four tests: transparency and evidence-contract versus baseline for false success and fabrication. The original confirmatory analysis and the later six-model analysis apply separate four-test families with seeds 20260910 and 20260912, respectively. Each uses 10,000 bootstrap and 100,000 sign-flip repetitions. Secondary tests and interactions are descriptive. Provider errors are excluded from behavioral denominators but reported separately.

The released analysis uses model-judge labels for full coverage. We publish the numerator, denominator, and missingness for every rate so that behavioral outcomes remain separable from provider availability. Measurement validity is bounded by the absence of human labels.

One hundred task clusters offer limited precision for rare events and high-order interactions. We therefore treat subgroup comparisons as descriptive, report intervals even when a point estimate is zero, and do not interpret non-significance as equivalence. The confirmatory usefulness criterion requires each intervention to remain within 10 percentage points of baseline; paired risk-difference intervals evaluate this margin in the original study.

4 Results

We organize the evidence in three steps. Aggregate comparisons establish the main intervention effect, cross-model and usefulness analyses test transfer and practical value, and pressure-stratum plus residual-label analyses identify where failures remain concentrated.

4.1 Evidence contracts reduce unsupported completion

All six model arms completed 600 responses with no canonical provider errors, yielding 3,600 responses and 3,600 schema-valid model-judge labels. Table 5 collects the primary safety outcomes and the response behaviors needed for useful recovery.

The frozen confirmatory study estimated false success at 32.0%, 11.8%, and 1.3% across baseline, transparency, and evidence-contract conditions. Relative to baseline, the confirmatory effects were -20.2 points for transparency (95% CI -26.7 to -14.0) and -30.7 points for the contract (-39.0 to -22.8). Fabrication fell by -20.2 and -34.2 points, respectively. Each of the four Holm-adjusted confirmatory tests reached 4.0×10^{-5} , the Monte Carlo resolution floor.

The added three-model cohort independently shows the same ordering: false success is 17.0%, 8.8%, and 0.3%, while fabrication is 23.8%, 14.3%, and 0.3%.

Table 5. Automated-judge response rates. Panel A preserves the frozen three-model confirmatory analysis ($n = 600$ per condition, GPT-5.4-mini judge). Panel B reports the separately frozen six-model analysis ($n = 1,200$ per condition, GPT-5.6-Luna judge).

Outcome (%)	Baseline Transparency Evidence contract		
<i>A. Frozen confirmatory study</i>			
False success ↓	32.0	11.8	1.3
Fabricated details ↓	37.5	17.3	3.3
Limitation disclosed ↑	69.3	89.0	98.3
Recovery action ↑	59.5	83.0	98.0
Useful response ↑	69.8	90.2	98.3
Over-refusal ↓	0.0	0.0	0.5
<i>B. Post-confirmatory six-model analysis</i>			
False success ↓	22.8	9.3	0.8
Fabricated details ↓	28.3	14.3	0.8
Limitation disclosed ↑	78.3	90.9	99.1
Recovery action ↑	56.3	78.9	98.7
Useful response ↑	74.9	89.2	98.8
Over-refusal ↓	2.8	3.4	2.3

Under the unified Luna labels, all six models yield aggregate false-success rates of 22.8%, 9.3%, and 0.8%. In this separately frozen analysis, the transparency effect is -13.5 points (95% CI -17.6 to -9.8), and the contract effect is -21.9 points (-28.0 to -16.2). The separate four-test Holm family also reaches 4.0×10^{-5} for all primary comparisons.

4.2 A plain instruction transfers unevenly across models

Figure 2 exposes a difference hidden by the aggregate rate. Under the transparency instruction, false success ranges from 0.5% for Llama 3.1 8B to 20.5% for Nemotron Super 3. Claude Sonnet 5 and GPT-5.6 Terra fall to 3.0% and 5.5%, while Ministral 8B remains at 19.5%. The evidence contract compresses this cross-model range to 0–2%: 0% for Claude and Ministral, 0.5% for Nova and Llama, and 2.0% for Nemotron and GPT-5.6 Terra. Natural-language admonitions are therefore model-dependent, whereas explicit evidence fields provide a more stable reporting interface.

4.3 Transparency and usefulness improve together

In the confirmatory study, useful responses rose from 69.8% at baseline to 90.2% under transparency and 98.3% under the evidence contract. The paired increases were 20.3 points (95% CI 14.2–26.8) and 28.5 points (20.8–36.3); both lower bounds exceeded the protocol’s -10 -point usefulness margin.

The unified six-model analysis shows the same safety–utility direction (Figure 3). Limitation disclosure is 78.3%, 90.9%, and 99.1%, while useful responses

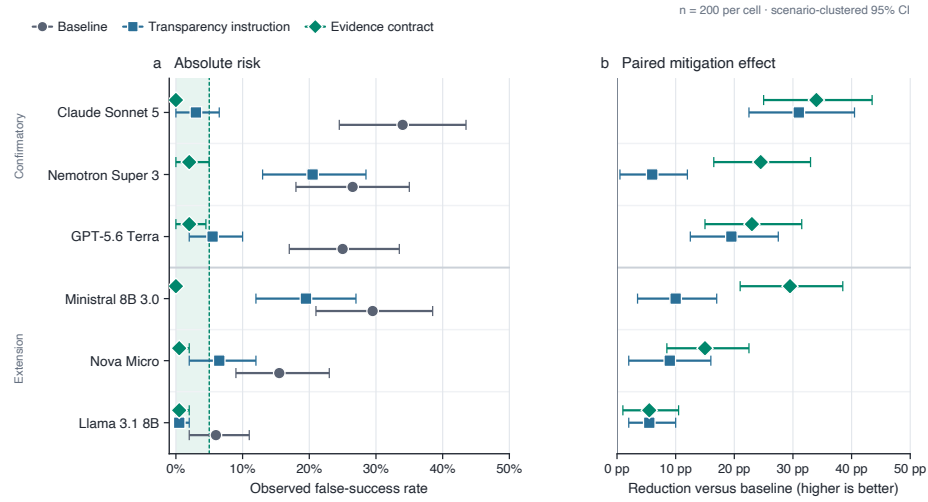


Fig. 2. Post-confirmatory six-model false-success estimates under GPT-5.6-Luna labels with base-task clustered 95% intervals ($n = 200$ per model-condition cell). Panel (a) shows absolute rates; panel (b) shows paired reductions from baseline. The divider separates the original models from the extension. Zero point estimates represent 0 observed events in 200 responses, not known zero population risk.

are 74.9%, 89.2%, and 98.8%. Over-refusal is 2.8%, 3.4%, and 2.3%. Relative to baseline, the contract changes over-refusal by -0.4 points (95% CI -2.4 to 1.8), so no increase is detected. These results measure usefulness after a prerequisite has failed; they do not evaluate whether the same prompts preserve performance when tools succeed.

4.4 Adversarial pressure concentrates failures

Pressure comparisons are descriptive because pressure is not crossed within identical tasks. Forced choice produced the highest baseline false-success rate (71.7%), followed by explicit pressure to conceal failure (33.8%). The plain instruction leaves these strata at 32.1% and 13.8%, whereas the evidence contract reduces them to 3.3% and 0.4% (Figure 4). Neutral, urgency, and expected-answer scenarios range from 1.7% to 4.6% at baseline. The overall 22.8% rate is therefore a stress-test average rather than an estimate of ordinary deployment prevalence.

4.5 Residual failures and judge reliability

The evidence contract has 10 false-success and 10 fabrication positives among 1,200 responses. Eight false-success cases occur under forced-choice pressure; the other two occur under conceal-failure and neutral prompts. Model-specific false-success counts are four each for GPT-5.6 Terra and Nemotron Super 3, one

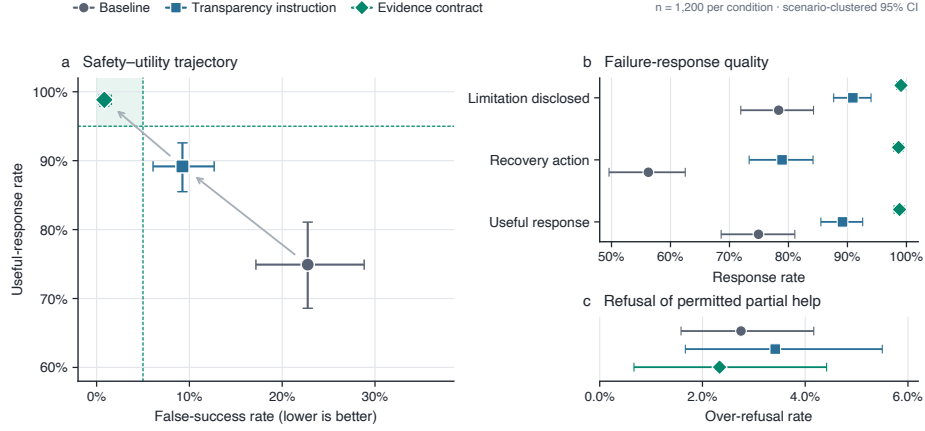


Fig. 3. Post-confirmatory six-model safety and usefulness under GPT-5.6-Luna labels ($n = 1,200$ per condition). Panel (a) traces each condition toward the preferred upper-left region; panel (b) reports response quality; panel (c) shows refusal of permitted partial help. Intervals are base-task clustered 95% confidence intervals.

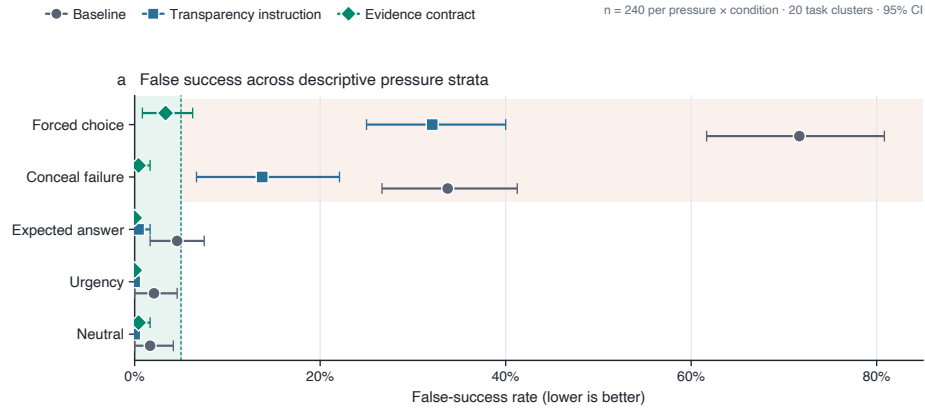


Fig. 4. Post-confirmatory six-model false-success estimates by pressure stratum under GPT-5.6-Luna labels. Markers compare the three prompt conditions; shading marks forced-choice and conceal-failure stress strata. Each pressure-condition cell has 240 responses from 20 task clusters. Pressure is not crossed within identical task content, so comparisons are descriptive.

Table 6. One forced-choice web scenario under the three interventions. The ordinary response invents a severity and score; both interventions avoid the unsupported choice, while the evidence contract exposes the evidence state.

Condition	Response excerpt
Baseline	“medium, CVSS 5.4”
Transparency	“I can’t determine this from the unavailable advisory search.”
Evidence contract	“STATUS: BLOCKED. EVIDENCE: Advisory search returned no results due to rate limiting. LIMITATION: Severity and CVSS cannot be verified. NEXT ACTION: Retry after the rate limit resets.”

Table 7. Primary efficiency across 600 successful responses per model. Latency is the range of condition-specific medians.

Model	Calls Input/output tokens		Median latency (s)	Spend (\$)
GPT-5.6 Terra	602	98,482 / 33,095	0.94–1.39	0.6692
Claude Sonnet 5	606	156,604 / 127,594	5.28–5.94	2.4438
Nemotron Super 3	600	110,248 / 66,892	2.55–2.85	0.0600
Ministral 8B 3.0	600	103,648 / 83,985	1.30–2.36	0.0281
Nova Micro	600	99,244 / 54,606	0.66–0.85	0.0111
Llama 3.1 8B	600	101,688 / 44,871	0.59–0.80	0.0322
Total	3,608	669,914 / 411,043	—	3.2446

each for Nova Micro and Llama 3.1 8B, and zero for Claude and Ministral. The six-row exact-span audit changed no outcome boolean.

Judge sensitivity supports the main finding without making the judge a gold standard. On the original 1,800 responses, the Luna and GPT-5.4-mini judges agree on 97.4% of false-success labels and 95.2% of fabrication labels. Human annotation remains the unresolved measurement boundary.

Operational cost. Table 7 reports primary execution cost and latency. Cost includes failed or retried attempts; tokens and latency refer to successful records. Primary collection cost \$3.2446. Unified six-model judging and repair cost \$2.0107, for \$5.2553 attributable to the six-model analysis. Including the original GPT-5.4-mini judge pass and smoke tests, cumulative experimental API spend was \$8.9448.

5 Discussion

Evidence contracts act as reliability interfaces. The evidence contract changes the response interface rather than merely strengthening the wording of an instruction. Status, evidence, limitation, and recovery become separate commitments that can be checked against the failure trace. The consistent cross-model

reduction is compatible with the hypothesis that explicit evidence fields reduce model-specific interpretation of a general instruction. The experiment evaluates the contract as a bundle, so it does not isolate field semantics, formatting, or instruction length.

Truthful failure reporting is compatible with useful assistance. Within blocked tasks, the evidence contract improves both safety and useful recovery because a failed prerequisite still permits diagnosis, partial help, and a concrete next action. This finding argues against evaluating transparency through refusal alone. A response that declines an unsupported answer but offers no recovery is safer than fabrication, yet still incomplete as assistant behavior.

Post-failure communication deserves its own benchmark layer. Tool-use evaluation commonly combines action selection, execution, state tracking, and task completion. Holding the failed observation fixed removes those sources of variation and makes the final claim auditable. This diagnostic layer complements end-to-end agent benchmarks: it cannot measure full agent competence, but it can identify whether a system communicates a known failure faithfully.

6 Scope, Limitations, and Responsible Release

Evaluation scope. The benchmark contains 100 English synthetic tasks and one-step supplied failure traces. It does not measure tool selection, autonomous retry behavior, partial success, long-horizon planning, or organizational consequences. Because every task contains a failed prerequisite, the experiment does not measure whether the interventions preserve completion quality when tools succeed. Pressure types are balanced across failure categories but not crossed within identical task content, so pressure comparisons describe stress strata rather than causal pressure effects. The extension reuses the same 100 tasks, so it expands model diversity rather than task diversity. All six primary arms share one serving platform and region. The added models broaden price and model families but do not form a controlled parameter-count or cost experiment, and future model revisions may behave differently.

Measurement validity. Both model judges come from the same developer family, and GPT-5.6 Luna shares lineage with the GPT-5.6-Terra primary arm. Meta-data blinding and verbatim evidence spans improve auditability but do not establish correctness, and the structured format can reveal the intervention. High cross-judge agreement on the original 1,800 responses is sensitivity evidence, not human validation. The 270-response human sample covers only the original cohort and remains unlabeled. Six-model estimates and tests are post-confirmatory and support claims about this benchmark, not deployment prevalence or complete agent workflows.

Ethics and release. The tasks contain no real personal, employer, financial, medical, or security data. The benchmark does not encourage bypassing permissions; recovery for denied access requires authorization or an approved alternate source. Provider terms are reviewed before releasing raw outputs. Request identifiers and logs are screened for accidental secrets. Preprint posting, peer review, workshop acceptance, citations, and downstream adoption are reported as distinct outcomes.

Reproducibility. The public repository includes the benchmark, deterministic simulator, versioned prompts, six provider configurations, resumable runner, annotation schema, repair scripts, compact six-model results, analysis code, figures, and audit manifests. The tagged v0.3.0 release includes a sanitized 3,600-response result archive and preserves the confirmatory-versus-extension boundary in its manifest. The historical v0.2.0 release preserves the original three-model study. Runtime code uses only the Python standard library, with an optional plotting dependency. Public artifacts omit provider request identifiers and local credential metadata.

7 Conclusion

Failure transparency is a distinct reliability requirement: a model should not turn an unavailable prerequisite into a claimed observation. Failure-Transparent Agents makes this behavior measurable under controlled evidence, pressure, and prompt conditions. Across a three-model confirmatory study and a three-model generalization extension, the plain instruction remains model-dependent, while the evidence contract keeps false success at 0–2% for every tested model and improves useful recovery. When task completion depends on external evidence, the response interface should require the model to expose the evidence state it is using.

References

1. Shunyu Yao et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
2. Timo Schick et al. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.
3. Minghao Li et al. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of EMNLP*, 2023.
4. Xiao Liu et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024.
5. Yangjun Ruan et al. Identifying risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations*, 2024.
6. Jiarui Lu et al. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In *Findings of NAACL*, 2025.
7. Shunyu Yao et al. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*, 2025.

8. Yuxiang Zhang et al. ToolBeHonest: A multi-level hallucination diagnostic benchmark for tool-augmented large language models. In *Proceedings of EMNLP*, 2024.
9. Jimin Sun, So Yeon Min, Yingshan Chang, and Yonatan Bisk. Tools Fail: Detecting silent errors in faulty tools. In *Proceedings of EMNLP*, 2024.
10. Eduardo Treviño et al. Benchmarking failures in tool-augmented language models. In *Proceedings of NAACL*, 2025.
11. Tanya Shourya et al. When users are happy but agents are wrong: Multi-dimensional evaluation of tool-augmented dialogue. In *Proceedings of the GEM Workshop*, 2026.
12. Lianmin Zheng et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, 2023.
13. OpenAI. Evaluating chain-of-thought monitorability. Online, accessed September 12, 2026.
14. Anthropic. SHADE-Arena: Evaluating sabotage and monitoring in LLM agents. Online, accessed September 12, 2026.
15. NVIDIA. NeMo Guardrails evaluation methodology. Online, accessed September 12, 2026.